

AI & ML Labwork

Commands for Numpy

* Import Numpy

import numpy as np

* Array creation

- np.array([1, 2, 3])
- np.ones((2, 2))
- np.linspace(0, 1, 5)
- np.linspace(0, 1, 5)
- np.zeros((3, 3))
- np.arange(0, 10, 2)
- np.eye(3)
- np.random.randn(1, 10, (2, 3))

* Array Properties

- arr.shape
- arr.size
- arr.ndim
- arr.dtype

* Array operations

- arr + 5
- arr * 2
- arr1 + arr2
- np.sum(arr)
- np.mean(arr)

* Indexing & slicing

- `arr[0]` → `arr[1,1]`
- `arr[1:4]` → `arr[1,2]`

* Reshaping

- `arr.reshape(2,3)`
- `arr.flatten()`

Pandas Command

* Import pandas

`import pandas as pd`

* Creating Series & DataFrame

- `pd.Series([10, 20, 30])`
- `pd.DataFrame({'A': [1, 2], 'B': [3, 4]})`

* Reading files

- `pd.read_csv("data.csv")`
- `pd.read_excel("data.xlsx")`

* Basic DataFrame command

- `df.head()` → `df.describe()`
- `df.tail()` → `df.shape`
- `df.info()` → `df.columns`

* Selection

- `df['A']`
- `df[['A', 'B']]`
- `df.loc[0]`

* Adding & Removing column.

→ `df['c'] = df['A'] + df['B']`

* Handling using values

→ `df.isnull()` → `df.dropna()`

→ `df.fillna(0)`

* Sorting & filtering

→ `df.sort_value('A')`

→ `df[df['A'] > 10]`

~~**~~ Commands for Matplotlib

* Import Matplotlib

`import matplotlib.pyplot as plt`

* Basic Plot

→ `plt.plot(x, y)`

→ `plt.show()`

* Types of graphs

→ `plt.scatter(x, y)`

→ `plt.hist(data)`

→ `plt.bar(x, y)`

→ `plt.pie(value)`

* Graph Customization

→ `plt.title("Graph Title")`

→ `plt.xlabel("x axis")`

→ `plt.ylabel("y axis")`

→ `plt.legend()`

→ `plt.legend()`

Seaborn Command

* Import Seaborn

import seaborn as sns

* Basic plots

→ sns.linelplot(x=x, y=y)

→ sns.barplot(x=x, y=y)

→ sns.scatter(x=x, y=y)

→ sns.histplot(data)

Q1 Plot a line graph showing marks of a student in 5 subject?

Ans import matplotlib.pyplot as plt

Subjects = ['Maths', 'Physics', 'Chemistry', 'English']

marks = [99, 98, 95, 97]

plt.plot(Subjects, marks, marker='o')

plt.title("Marks")

plt.xlabel("Subjects")

plt.ylabel("Marks of student")

plt.grid()

plt.show()

Q2 Plot a heatmap to show correlation b/w feature?

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

data = { 'Maths': [89, 98, 99],

'Physics': [97, 96, 97],

'Chemistry': [99, 100, 98] }

```
df = pd.DataFrame(data)
sns.heatmap(df.corr(), ant=True)
plt.title("Correlation heatmap")
plt.show()
```

Q8 Histogram for age distribution of students

```
import matplotlib.pyplot as plt
age = [19, 20, 21, 22, 23, 24, 19, 20]
plt.title("Age Distribution")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

Q9 Create a dataset of health care :-

```
Patient name, ID, Age, Gender, Blood
group, Disease, B.P, height, weight, income
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
data = {'Name': ['John', 'Suzene', 'Tom',
                  'Jerry', 'I', 'Jahre',
                  'Rohan', 'Kini']
```

```
    'Id': [101, 102, 103, 104, 105, 106,
           107, 108],
```

```
    'Age': [22, 19, 18, 45, 25, 18,
            22, 35],
```

```
    'gender': ['M', 'M', 'F', 'F', 'M', 'M',
               'M', 'F']
```

```
    'Blood group': ['A+', 'AB+', 'B+', 'B-',
                    'O+', 'A-', 'B-', 'B+']
```


'weight': [72, 60, 50, 100, 70, 120,
75, 80],

~~'income'~~ : [10000, 20000, 30000, 40000,
50000, 60000, 70000, 80000]

d-frame = pd.DataFrame(data, index =

CR1, R2, R3, R4, R5, R6, R7, R8))

print (d-frame)

plt.figure()

plt.bar (name, weight)

plt.xlabel ("Name")

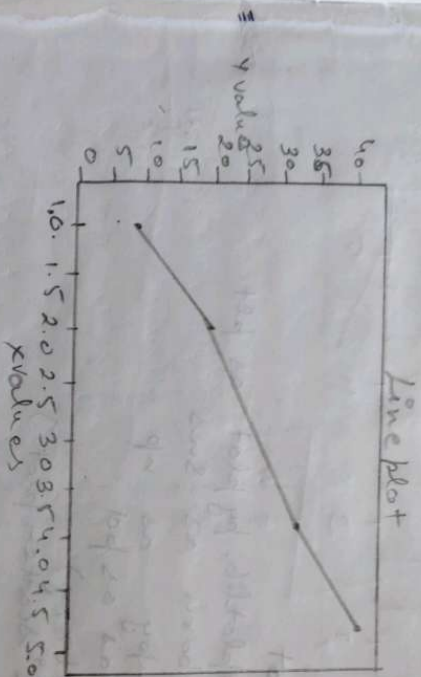
plt.ylabel ("weight")

Output

Name	ID	Age	Gender	Blood	income weight
Jalin	101	22	M	A+	72
Suzene	102	19	M	B+	80
Tome	103	18	F	AB+	60
Jeery	104	45	M	B-	100
Take	105	25	M	A+	70
J	106	28	M	A-	50
Jichang	107	22	F	O+	80
Kive	108	35	F	B+	60

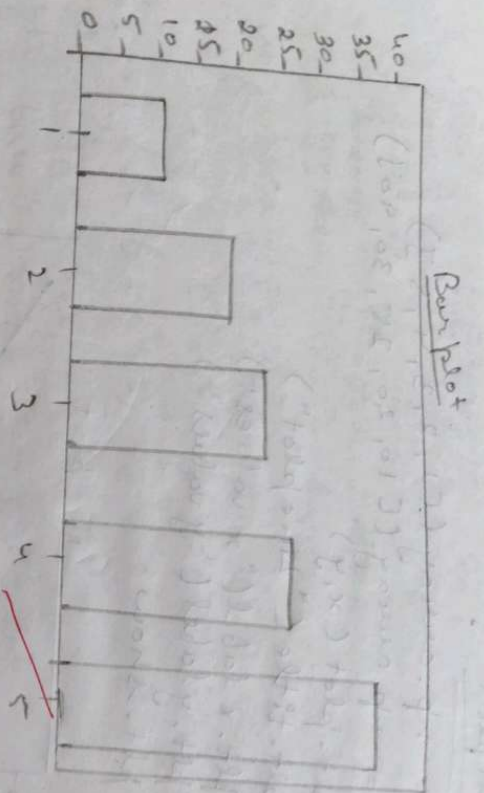
Matplotlib :

```
x = np.array([1, 2, 3, 4, 5])  
y = np.array([10, 20, 25, 30, 40])  
plt.plot(x, y)  
plt.title("Line plot")  
plt.xlabel("x values")  
plt.ylabel("y values")  
plt.show()
```



Bar Graph

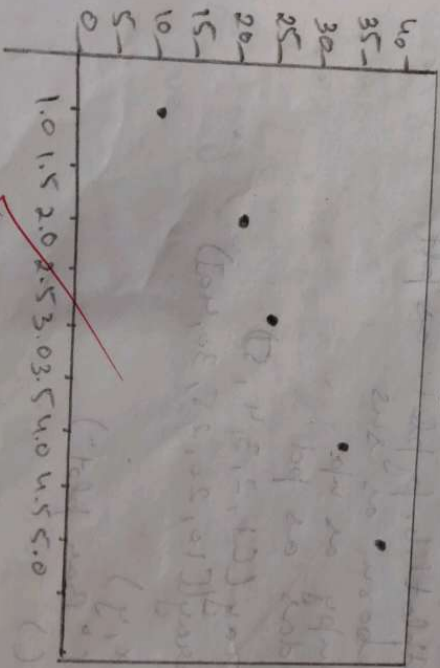
```
import matplotlib.pyplot as plt  
import seaborn as sns  
import numpy as np  
import pandas as pd  
x = np.array([1, 2, 3, 4, 5])  
y = np.array([10, 20, 25, 30, 40])  
plt.bar(x, y)  
plt.title("Bar plot")  
plt.show()
```



Scatter plot

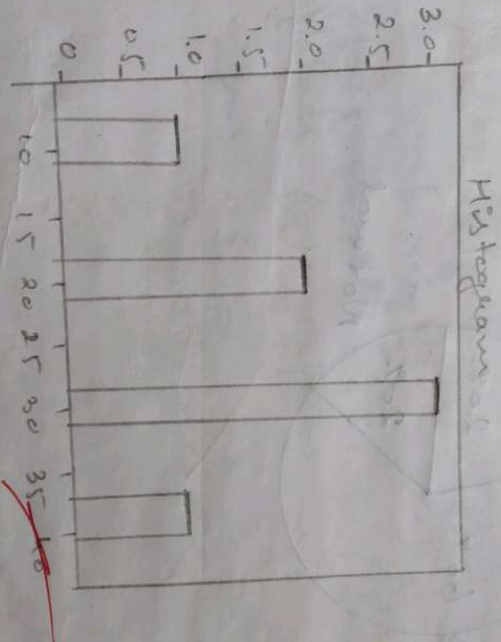
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
plt.scatter(x, y)
plt.title("Scatter plot")
plt.show()
```

Scatter plot



Histogram:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
data = np.array([10, 20, 30, 30, 30, 40])
plt.hist(data)
plt.title("Histogram")
plt.show
```

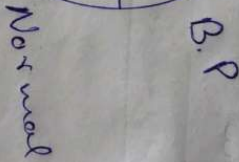


Pie chart:

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd
labels = ["A", "B", "C", "D"]
sizes = [30, 40, 30]
plt.pie(sizes, labels=labels, autopct =
%1.1f)
plt.title("Pie chart")
plt.show
```

303
22A
22C
22P

Diagnosis



02/02/26

Def Depth first Search

def dfs (graph, start, visited = None)

if visited is None:

visited = set()

print (start, end(''))

visited.add (start)

for neighbour in graph [start]

if neighbour not in visited

dfs (graph, neighbour, visited)

Values for graph

graph = {

'A': ['D', 'E'],

'B': ['A', 'B'],

'C': ['C', 'B'],

'D': ['J'],

'E': ['J']

}

Output → B A D E

- Graph
- BFS and DFS
- Why studying BFS and DFS?
- Real life application?

Answers

Ans 1 ~~Graph~~ Graph is a way to show information visually. It uses points and lines to represent how two or more things are related. One thing is shown in horizontal axis and another on the vertical axis.

- Graph in CS is a data structure used to represent relationships b/w things.

⇒ A graph is made of

- Vertices (Nodes): the items
- Edges (links): the connections b/w items

Ans 2 DFS (Depth first Search)

- DFS goes as deep as possible before coming back.

Steps

- 1 → Start at a node.
- 2 → Visit one neighbour.
- 3 → Keep going deeper until there is nowhere left to go.
- 4 → Then backtrack and try another path.

uses

- checking connectivity
- Detecting cycles
- Solving puzzles like mazes
- Topologic sorting

Data structure used: Stack (or recursion which is just a stack in disguise)

BFS (Breadth First Search)

- BFS goes level by level

- 1 → Start at a node
- 2 → visit all its neighbors first.
- 3 → Then visit neighbors of those neighbors

uses

- Finding the shortest path in an unweighted graph.
- level order traversal
- Broadcasting system

Data structure used: Queue (because order actually matters here)

~~D~~ DFS

BFS

- Goes deep first
- Uses stack
- May not find shortest path
- Less memory sensitive
- Goes wide first
- Uses queue
- Find shortest path
- More memory

Ans 3 Why BFS and DFS :

- 1 → Searching problem spaces.
AI has to explore many possible states.
 - 2 → Foundation for advanced algorithms
 - 3 → Tree structure in ML
Decision, parse, syntax trees all need traversal.
 - 4 → Graph based learning
 - 5 → Interview and logical thinking
- DS knowledge

Ans 4 Real life application :-

* DFS -

- 1 → File and Folder search
- 2 → Website crawling
- 3 → cycle Detection

* BFS -

- 1 → Social Media (Friends suggestion)
- 2 → Shortest Path (GPS & maps)
- 3 → Minimum Steps Problems

Ans:

Q Design and Develop DFS algorithm for the following?

(i) → Static Graph consisting of 6 node and 10 edges where the graph is stored in dictionary



1. A → B
2. A → C
3. B → D
4. B → E
5. C → D
6. C → E
7. D → F
8. E → F
9. F → D
10. F → E

(ii) → Create another graph dynamically where the value of graph is given by user

(iii) → Accept the root node and goal node

(iv) → Display the path from root node and goal node

Ans (i) graph = { 'A': ['B', 'C'],

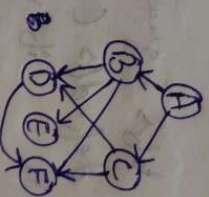
'B': ['D', 'E', 'F'],

'C': ['D', 'E', 'F'],

'D': ['E', 'F'],

'E': ['F'],

'F': ['D'], }



def dfs(graph, start, visited=None):

if visited is None:

visited = set()

print(start, end='')

visited.add(start)

for neighbour in visited graph[start]:

if neighbour not in visited:

dfs(graph, neighbour, visited)

Output → A B D E

Ans graph = { 'A': ['B', 'C'],
 'B': ['D', 'E', 'F'],
 'C': ['D', 'F'],
 'D': ['E', 'F'],
 'E': ['F'],
 'F': [] }

```
def bfs (graph, Node)
    visited = []
    queue = []
    visited.append(Node)
    queue.append(Node)
    while queue:
        m = queue.pop(0)
        print(m)
```

for n in graph[m]:
 if n not in visited:

visited.append(n)

bfs (graph, 'B')
 out → B D F

Ans 2 def bfs (graph, start, visited = 'none'):
 if visited is none:

visited = set()

print (start, end = " ")

visited.add (start)

for neighbour in graph[start]:

if neighbour not in visited:

bfs (graph, neighbour, visited)

def (graph, 'A')

Output → A B F

Q Normalization and standardization?

At Mike Data of AmNave, Age, Height, Salary.

* from sklearn.preprocessing import MinMaxScaler

Scaler = MinMaxScaler()

df[['Age', 'Salary', ']] = Scaler.fit_transform(df[['Age', 'Salary']])

* from sklearn.preprocessing import StandardScaler

Scaler = StandardScaler()

df[['Age', 'Salary']] = Scaler.fit_transform(df[['Age', 'Salary']])

A* Code

```
import networkx as nx
graph = nxk()
```

while True:

```
choice = int(input("Press 1 to insert & 2 to exit"))
node1 = input("Enter node 1: ")
node2 = input("Enter node 2: ")
weight = int(input("Enter weight b/w node 1 & 2: "))
graph.add_edge(node1, node2, weight)
```

else:

break

Code

```
start = input("Enter start node").strip().upper()
```

```
goal = input("Enter goal node").strip().upper()
```

```
heuristic_values = {}
```

```
heuristic_value[goal] = 0
```

```
for node in graph.nodes():
```

```
if node != goal:
```

```
heuristic_val = int(input("Enter heuristic value for node to goal"))
```

```
heuristic_value[node] = heuristic_val
```

```
heuristic_value[goal] = 0
```

```
def get_heuristic(u, v):
```

```
return heuristic_value[u]
```

```
paths = nxk.astar_path(graph, start, goal, lambda u, v:
```

```
heuristic_value[u, v])
```

```
→ join(paths
```

```
total_cost = nxk.astar_path_length(graph, start, goal,
```

```
heuristic_value)
```

print (total-cost)
 print (heuristic-values)
 print (graph.edges (data=True))

Input

2 to exist)) Press 1 to insert and 2 to exist = 1
 upper
 open
 1 & node
 weight)
 Press 1 to insert and 2 to exist = 1

Enter node 1 = B
 Enter node 2 = C
 Enter weight b/w node 1 and node 2 = 8 2

node 1 = C
 node 2 = D
 weight = 2

node 1 = D
 node 2 = E
 weight = 2

node 1 = E
 node 2 = F
 weight = 1

Enter start node = A

Enter goal node = D
 heuristic value from node 1 to goal = 5
 6
 8
 4
 2

heuristic
 23: 1)

print (weight)

enter goal
 at weight)

GraphEdge = [(A, B, 3, {weight: 2, 3}), (B, C, 2, {weight: 3, 3}), (C, D, 1, {weight: 2, 3}), (D, E, 1, {weight: 2, 3}), (E, F, 1, {weight: 1, 3})]

Heuristic values: {D: 0, A: 5, B: 6, C: 8, E: 4, F: 2}

Total cost: 3

df = pc
from
scale
df CC
print
Output
0 1 2 3 4

2) Create a DataFrame and perform below task:

- 1) Standardization
- 2) Normalization

→ import pandas as pd

data = {'Name': ['Jack', 'John', 'Mridul', 'Bob', 'Preet'],

'Age': [23, 34, 24, 26, 27],

'Height': [180, 183, 167, 176, 195],

'Income': [100000, 50000, 130000, 120000, 150000]}

df = pd.DataFrame(data)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

df[['Age', 'Income']] = scaler.fit_transform(df[['Age', 'Income']])

df[['Age', 'Income']]

print(df)

Output

	Name	Age	Height	Income
0	Jack	-0.893325	190	-0.293610
1	John	1.836279	183	-1.761661
2	Mridul	-0.893325	167	0.587220
3	Bob	-0.148888	176	0.293610
4	Preet	0.099258	195	1.174440

5

Scales = MinMax Scales

$$df[CC'Age', 'Income'] = scale_fit_transform(df[CC'Age', 'Income'])$$

Paint (alg)

Output:

	Name	Age	Height	Income
0	Jack	0.000000	180	0.8
1	John	0.407407	183	0.6
2	Mridul	1.000000	167	0.8
3	Bob	0.111111	176	0.7
4	Peeti	0.148148	143	1.0

Linear Regression code

from sklearn.model_selection import train_test_split

import numpy as np

from sklearn.linear_model import LinearRegression

import matplotlib.pyplot as plt

from sklearn.metrics import mean_absolute_error, mean_squared_error

x = np.array([1, 2, 3, 4, 5])

y = np.array([2, 3, 5, 4, 6])

model = LinearRegression()

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

model.fit(x_train, y_train)

y_pred = model.predict(x_test)

print(model.coef_)

print(model.intercept_)

print("Mean Absolute Error:", mean_absolute_error(y_test, y_pred))

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))

plt.scatter(x, y, color='blue')

plt.plot(x_test, y_pred, color='red')

plt.xlabel("Independent variable (x)")

plt.ylabel("Dependent variable (y)")

plt.show()

O/P → Prediction box: [12.]

slope (coefficient): [2.]

Intercept: 1.7763568394002505e-15

mean absolute error: 8.8174197001252e-11

mean square error: 7.88664052210118e-31